Week 03

조건문과 반복문으로 흐름 제어

AI활용코딩연습(ZBE135)

이영호 교수 · 국립목포대학교 컴퓨터학부

지난주 돌아보기

```
h, rest = divmod(7384, 3600)        # 2, 184
round(2.5)                          # 2  (은행가 반올림)
"202612034"[4:6]                    # '12'
each = total // people // 100 * 100
assert each * (people - 1) + last == total
```

> AI에게는 **규칙과 검증 조건**을 함께 알려 준다

오늘의 목표

1. `if / elif / else`와 비교 · 논리 연산자로 여러 조건을 판정한다
2. **조건의 순서**와 **경계값**이 결과를 바꾼다는 것을 확인한다
3. `for`, `range`, `enumerate`, `while`, `break`, `continue`를 쓴다
4. 반복 패턴 4가지로 **장학 기준 판정기**를 만든다
5. AI 코드를 **경계값 표**로 검증한다

수업 흐름

구간	시간	활동
도입 · 개념	20분	조건 순서, 경계값, 반복 패턴
안내 실습	45분	노트북 1~2절
독립 · 짝 실습	35분	체감 온도 · 구구단 바꾸기, 장학 판정기
공유 · 점검	20분	경계값 표로 AI 코드 검증 결과 공유

이번 주 Colab 실습 열기

비교 · 논리 연산자

비교	의미	논리	의미
`==` `!=`	같다 / 다르다	`and`	둘 다 참
`<` `<=`	미만 / 이하	`or`	하나라도 참
`>` `>=`	초과 / 이상	`not`	뒤집기

```
score = 85
80 <= score < 90              # True  (연쇄 비교)
score >= 90 or score == 85    # True
```

if · elif · else

```
if temperature >= 33:
    print("폭염 경보")
elif temperature >= 28:
    print("더움")
elif temperature >= 10:
    print("활동하기 좋음")
else:
    print("추움")
```

- 위에서부터 **처음 참인 조건 하나만** 실행
- `elif`가 여러 개여도 실행되는 블록은 **딱 하나**

예상해 보기: 조건의 순서

```python
def grade_v2(score):
    if score >= 70:
        return "C"
    elif score >= 80:
        return "B"
    elif score >= 90:
        return "A"
    else:
        return "F"

grade_v2(95)   # ?
```

정답: **"C"** — `95 >= 70`이 먼저 참 → A는 영원히 나오지 않음

> **엄격한(좁은) 조건을 먼저** 쓴다

참 · 거짓으로 취급되는 값

거짓처럼	참처럼
`0`, `0.0`	`7`, `-1`
`""` 빈 문자열	`"목포"`
`[]` 빈 리스트	`[1, 2]`
`None`	

```
name = "   "
if not name.strip():
    print("이름이 비어 있습니다.")
```

중첩 조건을 평평하게

```
# 중첩: 읽기 어렵다
if year >= 1:
    if credits <= 21:
        if not has_debt:
            return "신청 가능"
        else:
            return "미납금 있음"
    else:
        return "학점 초과"
else:
    return "학년 오류"
```

```
# 평평: 안 되는 경우부터 return
if year < 1:
    return "학년 오류"
if credits > 21:
  return "학점 초과"
if has_debt:
  return "미납금 있음"
return "신청 가능"
```

early return: 예외 상황을 먼저 걸러 내면 핵심 로직이 맨 아래에 남는다

반복문 한눈에

형태	언제
`for x in 모음:`	원소를 하나씩
`for i in range(1, 6):`	정해진 횟수 (1~5, 끝은 **미만**)
`for i, x in enumerate(모음, 1):`	번호와 원소 함께
`while 조건:`	횟수를 모를 때

```
list(range(0, 20, 5))      # [0, 5, 10, 15]
list(range(10, 0, -3))     # [10, 7, 4, 1]
```

반복 패턴 4가지

패턴	시작	반복 안에서
누적	`total = 0`	`total += x`
개수 세기	`count = 0`	`if 조건: count += 1`
최댓값	`best = 첫 원소`	`if x > best: best = x`
모으기	`result = []`	`if 조건: result.append(x)`

> 대부분의 반복 문제는 이 네 패턴의 **조합**

패턴 조합 예

```
scores = [72, 95, 88, 59, 100, 67, 81]
total = passed = 0
best = scores[0]
low = []
for s in scores:
    total += s
    if s >= 60:
        passed += 1
    if s > best:
        best = s
    if s < 70:
        low.append(s)
# 평균 80.3점, 합격 6명, 최고 100점, 70점 미만 [59, 67]
```

break와 continue

```
readings = [21.5, 22.0, -999, 23.1, 22.8, 85.0, 23.0]
for t in readings:
    if t == -999:           # 센서 오류값
        continue            # 이번 것만 건너뛰기
    if t > 50:              # 이상 고온
        break               # 반복 전체 중단
    print("정상 측정:", t)
```

출력: 21.5 → 22.0 → 23.1 → 22.8 → **중단**

while과 안전장치

```
goal, monthly = 2_000_000, 300_000
balance = month = 0
while balance < goal:
    month += 1
    balance += monthly
    if month > 120:                # 안전장치
        print("도달 불가")
        break
print(month, "개월")               # 7 개월
```

- `monthly = 0` 이면? 조건이 영원히 참 → **무한 반복**
- Colab에서 멈추지 않으면 ■ 정지 버튼

실습 1 · 2 (Level 1 → Level 2)

Level 1 따라하기

- 조건 순서 실험 (`grade_v1` vs `grade_v2`)
- 반복 패턴 4가지, `break` / `continue` , `while`

Level 2 바꾸기

- 체감 온도: 경계값 `33` , `28` , `27.9` 로 바꾸고 "한파 주의" 조건 추가
- 구구단: 3단 · 7단, **짝수 곱만** 출력 (`continue`)

실습 3 (Level 3): 장학 기준 판정기

장학	조건	우선순위
성적장학	평점 4.0 이상 **and** 이수학점 15 이상	1
성실장학	출석률 95% 이상 **and** 평점 3.0 이상	2
해당없음	그 외	–

```
def scholarship(gpa, credits, attendance):
    if gpa >= 4.0 and credits >= 15:
        return "성적장학"
    if attendance >= 95 and gpa >= 3.0:
        return "성실장학"
    return "해당없음"
```

경계값으로 검증

gpa	학점	출석	기대	확인하는 것
4.0	15	0	성적장학	경계 그 자체
3.99	15	0	해당없음	바로 아래
4.0	14	0	해당없음	학점 바로 아래
3.0	0	95	성실장학	성실 경계
4.5	21	100	성적장학	우선순위

`>=` 와 `>` 의 차이는 **경계에 있는 값**으로만 드러난다

여러 학생 판정 + 요약

```
merit = diligence = none = 0
for i, (name, gpa, credits, att) in enumerate(students, 1):
    result = scholarship(gpa, credits, att)
    if result == "성적장학":
        merit += 1
    elif result == "성실장학":
        diligence += 1
    else:
        none += 1
    print(f"{i:<5}{name:<6}{gpa:>6.2f}{credits:>6}{att:>6}%  {result}")

assert merit + diligence + none == len(students)   # 빠진 학생 없음
```

AI가 준 학점 함수

```
def grade_ai(score):
    if score > 90:   return "A"
    elif score > 80: return "B"
    elif score > 70: return "C"
    elif score > 60: return "D"
    else:            return "F"
```

`95 → A`, `85 → B`, `75 → C`, `30 → F` ... **잘 되는 것처럼 보인다**

경계값 표로 검증하기

점수	기대	AI 결과	
90	A	B	❌
80	B	C	❌
60	D	F	❌
59	F	F	✅
−1	오류	F	❌
101	오류	A	❌

경계 그 자체 · 바로 아래 · 범위 끝 · **잘못된 입력**

고친 코드

```python
def grade_fixed(score):
    if score < 0 or score > 100:   # 잘못된 입력 먼저
        return "오류"
    if score >= 90:                # > 를 >= 로
        return "A"
    if score >= 80:
        return "B"
    if score >= 70:
        return "C"
    if score >= 60:
        return "D"
    return "F"
```

> AI는 **"이상"과 "초과"** 를 자주 헷갈린다 → 프롬프트에 경계를 적고, 경계로 검증

흔한 실수 모음

실수	증상	해결
넓은 조건을 먼저	일부 결과가 안 나옴	엄격한 조건 먼저
`>` / `>=` 혼동	경계에서만 틀림	경계값 표
`if x == 1 or 2:`	항상 참	`x == 1 or x == 2` / `x in (1, 2)`
`range(1, 10)`로 10까지 기대	9에서 끝	끝은 **미만**
`while` 조건 불변	무한 반복	안전장치 · 최대 횟수

이번 주 과제 (10점)

문제	내용	배점
1	`grade(score)` A+~F, 오류 + **내 경계값 테스트 4개**	2
2	`mokpo_game(n)` 목포대학 게임	2
3	`scholarship` 4단계 + `count_scholarships` 집계	3
4	`months_to_goal` 저축 계산기, 무한 반복 막기 (AI 협업)	3

문제 4: 프롬프트에 **무한 반복이 생기는 경우**를 적었는지 기록

자기 점검

1. `grade_v2(95)`가 `"C"`인 이유는?
2. 점수 규칙이 "60점 이상 합격"일 때 꼭 넣어야 할 테스트 값 4개는?
3. `if x == 1 or 2:`가 항상 참인 이유는?
4. 최댓값을 찾을 때 `best = 0`으로 시작하면 생길 수 있는 문제는?
5. `while`에 안전장치가 필요한 상황의 예는?

정리와 다음 주

오늘	핵심
조건문	엄격한 조건 먼저, 연쇄 비교, early return
반복	`range` 끝은 미만, `enumerate`, `while` 안전장치
패턴	누적 · 개수 · 최댓값 · 모으기
검증	경계값 표: 경계 · 바로 아래 · 범위 끝 · 잘못된 입력

다음 주 · Week 04 리스트 · 튜플 · 집합 · 딕셔너리 — 학생 성적 레코드 분석